School of Electronic Engineering and Computer Science

Final Report

Programme of study:
BSc (Hons)
Mathematics and Computer Science

Project Title:

Supervisor:
Piyajeth Wijetunge

Student Name:
Jandry Guadir Rodriguez

Final Year
Undergraduate Project 2024/25

Queen Mary University of London

Date: 05/11/2025

Abstract

Amateur football coaches at the grassroots and university level are frequently left to manage their squads using manual logs, group chats, and non-specialised software. While professional clubs invest millions into data-driven performance systems, the amateur game lacks any accessible, intelligent tool that supports the coaching side of team management. This project addresses that gap through the design and development of Coach AI, an iOS first but eventually cross platform mobile application built with React Native and Supabase. Driven by personal experience managing a university football team through WhatsApp groups and scattered notes, the application focuses on reducing the administrative burden that volunteer coaches face weekly. A survey of over 20 amateur coaches and players informed the project's final requirements, which were prioritised using MoSCoW analysis and refined through pre- and post-survey comparison. The end application provides core features including team scheduling, player availability tracking, simple wellness questionnaires, and AI generated insights powered by a Pythonic middleware layer. Testing was conducted across component, integration, performance, and user acceptance levels. The evaluation shows that Coach AI successfully consolidates key coaching tasks into a single, focused tool, addressing a gap area of the current amateur app market where existing solutions either specialise in administration alone or sit behind prohibitive paywalls.

Contents

Chapter 1: Introduction

1.1 Background and Motivation

The word “amateur”, in the respect to coaching, refers to a person who does something (such as a sport or hobby) for pleasure and not as a job (Zeylurt, 2016). In England alone, grassroots sport is a massive part of everyday; Sport England’s Active Lives survey reports that almost 31 million adults are physically active, with amateur sports mostly managed by volunteer coaches and organises who receive no formal training or wages (Sport England, 2026). Football sits at the centre of this landscape as the country’s most popular team sport, yet the technology available for volunteers who run this hasn’t evolved at the same rate as its professional counterpart.

From modern stadiums utilised as year-round entertainment to financial institutions investing heavily in clubs, the extensive wealth growth of professional football teams has enabled the aggressive inclusion of private technology. For top clubs, investment in technology is no longer a luxury but a requirement to maintain a competitive edge; for instance, the Guardian (2025) reported that Brentford FC invested approximately £16 million into their research department between 2022 and 2024 alone. Even though Brentford is a forefront runner for use of technology, their investment highlights a broader industry trend where larger sporting organisations are adopting a technological approach to every facet of the game. At the elite level, the use of technology for player monitoring is standard practice. A survey of 41 elite football clubs found that every club used GPS tracking and heart-rate monitoring during training, while 68% also used wellness questionnaires to monitor player readiness (Akenhead, 2016).

Despite the clear benefits of data-driven coaching, there is a notable absence of any intelligent tools within the amateur environment. The technology that exists at the professional level has not been adapted down; grassroots coaches are left to bridge the gap themselves using tools not designed for their job.

My motivation for this project comes from years of personal experience playing in and being around amateur football, where I have witnessed first-hand the disconnect between available technology and daily practice. While elite teams use wearable GPS vests and real-time analytics, amateur coaches are frequently left to manage their squads’ progress using manual logs, paper diaries, or non-specialised software like Excel or Notes (iOS). This manual approach is usually insufficient for the complexities of modern team management; a study of football coaches found that while nearly 94% reported monitoring training loads, the vast majority relied on basic, non-automated diaries to do so (Bokůvka, et al., 2025). This creates a significant "administrative tax" on coaches who, much like those at Queen Mary Football Club, must balance their passion for the game with full-time academic or professional commitments. As a result, there is a critical need for an assistant that brings the power of easy to use technology to non-professional environments, supporting coaches in making better-informed decisions while significantly reducing their weekly administrative burden.

1.2 Problem Statement

There is a clear gap between the professional clubs and the amateur grassroots game when it comes to the meaninful use of technology. At the amateur level, a coach’s role typically extends far beyond tactical instruction. Volunteers are weighed down by a high-pressure

administrative and operational burden, often filling roles that professional clubs distribute across dedicated staff. From tracking player attendance and monitoring performance to maintaining constant communication and selecting team lineups, the multiple facets of coaching are immensely time-consuming for volunteers who often lack formal training and have limited time to spare.

Commercial applications typically assist with just the administration. They often focus on scheduling and messaging, with little offered regarding performance analytics and intelligent decision support. Most current solutions do not leverage the data that coaches can and already do collect. They offer little in the production of intelligent insights and data focused suggestions to inform management decisions. This lack of intelligent insight means coaches need to expend more energy and time to identify patterns such as declining attendance, player fatigue, or inconsistent performance. These issues are especially prevalent at the university and grassroots levels, where teams are typically run by volunteers with limited time and resources. The absence of a simple tool that combines administrative support with intelligent performance analysis represents the core problem this project aims to address.

1.3 Aim

The primary aim for this project is to design and develop a mobile application that reduces the admin load on amateur football coaches, particularly those with limited time. The application will serve as a practical, data-driven assistant by combining team management features like scheduling and attendance tracking with intelligent performance insights derived from player questionnaire data.

1.4 Objectives

- Complete a thorough literature review to understand the management challenge amateur coaches face, the current solutions to it and how our solution can be better over the others.
- Conduct primary research to feed the project's requirements and target features.
- Develop a backend system capable of storing and processing all the team related data like training attendance and player's statistics such as goals and assists.
- Add features to the application like:
    - User authentication for players and coaches
    - Attendance log for events
    - Team creation – Adding players
    - Review questionnaires and analysis of data
- Evaluate the system through functional testing and user feedback from real coaches (QMFC and amateur clubs).

1.5 Research Questions

- What are the most time-consuming aspects to the role of an amateur football coach?
- What are current strategies employed to ease the stressors of the volunteer role?
- How can AI-driven analytics improve the efficiency and decision-making process of amateur sports coaches?

1.6 Report Structure

This report is divided into chapters, with each subheading added to further partition information and research for easier digestion. Their general content descriptions are detailed below.

Chapter 2: Literature Review – Through the thoughtful collation of research papers, articles and reports, this chapter explores the nuances of the problem. It also investigates why a solution is needed and how a viable resolution can be formulated. Looking at contemporaneous solutions, the advantages and disadvantages are scrutinised to gauge the technological gap to produce the foundation of a meaningful application.

Chapter 3: Methodology – This chapter is used to create the set of questions in the primary research. It analyses the questions' purpose and how survey respondents were identified.

Chapter 4: Requirements and Design – Directed by previous collected primary data, this chapter lists requirements which are used to shape the application's features and to create a standard to analyse how well the final app has been implemented. This part is also where the initial designs and use case diagrams are included.

Chapter 5: Implementation - This chapter details how the application was developed. It shows how errors were resolved and explaining any key app changes.

Chapter 6: Testing – This section allows us to check the completeness of the app, putting it under scrutinising tests to ensure the app has no unexpected errors during real use.

Chapter 7: Evaluation - This chapter considers how closely the final product followed the set-out requirements and analyses the impact of earlier chapters.

Chapter 8: Conclusion – This part wraps up the project by delving into the efforts produced and personal gripes developed. It also recognises the possible future development.

Appendices – Additional tables that do not fit well moved are moved to this section.

Chapter 2: Literature Review

2.1 The problem

2.1.1 The operational difficulty of under-resourced amateur football coaching

The landscape of grassroots football in the United Kingdom is currently defined by a profound paradox: while participation is at an all-time high, with over 2.2 million active players recorded in 2024 (Sport England, 2025), the infrastructure that supports the amateur scene is increasingly fragile. With a contribution of over £1 billion annually to the London economy alone (Consultancy.uk, 2021), the social and economic impact of grassroots is immense. Despite this, the operational reality for the workforce underpinning this amateur system is defined by systemic strain and resource scarcity (Taylor, 2025). At the centre of this struggle is the amateur coach, a figure forced to balance tactical development with an overwhelming administrative and financial burden. Research indicates that these struggles are systemic, driven primarily by "time poverty" and a significant "administrative tax" on volunteer time. According to Spond (2019), nearly 70% of grassroots coaches spend at least two hours per week on purely organisational tasks, while 24% dedicate over seven hours—equivalent to a full working day—to administration before even reaching the pitch. Spond also reports around a third of coaches consider the amount of time involved as a primary challenge in their role. This hidden work, including RSVP tracking and player correspondence, is a leading cause of burnout (Carrol, 2023).

This operational difficulty is exacerbated by a notable "data reporting" gap between professional and amateur tiers. While top organisations produce daily reports to support player availability and training planning, lower-tier clubs often abandon data collection due to staffing constraints and the expert-led nature of traditional analytics. This leads to analytics fatigue, where the head coach abandons data tracking when it becomes too burdensome.

2.2 Existing Solutions

Understanding the current diverse market of applications tailored to amateur coaches will determine the set of useful features or digestible design that may be included in the final product.

2.2.1 Spond

Spond is a widely adopted team and club management platform frequently utilised within UK grassroots sports. It is primarily designed to simplify the managerial burdens that arise from organising and sharing information for any type of sports-related group activity. The platform claims to save administrators an average of 2.2 hours of work per week (Spond, 2025) by streamlining communication and logistics. The application architecture supports a diverse range of user types, including coaching staff, players, and guardians. A notable feature is the integration of parental accounts, which allows guardians of youth players to manage attendance for upcoming events and facilitate the payment of club fees. As an established, independent company, Spond provides comprehensive documentation and robust support for its user base.

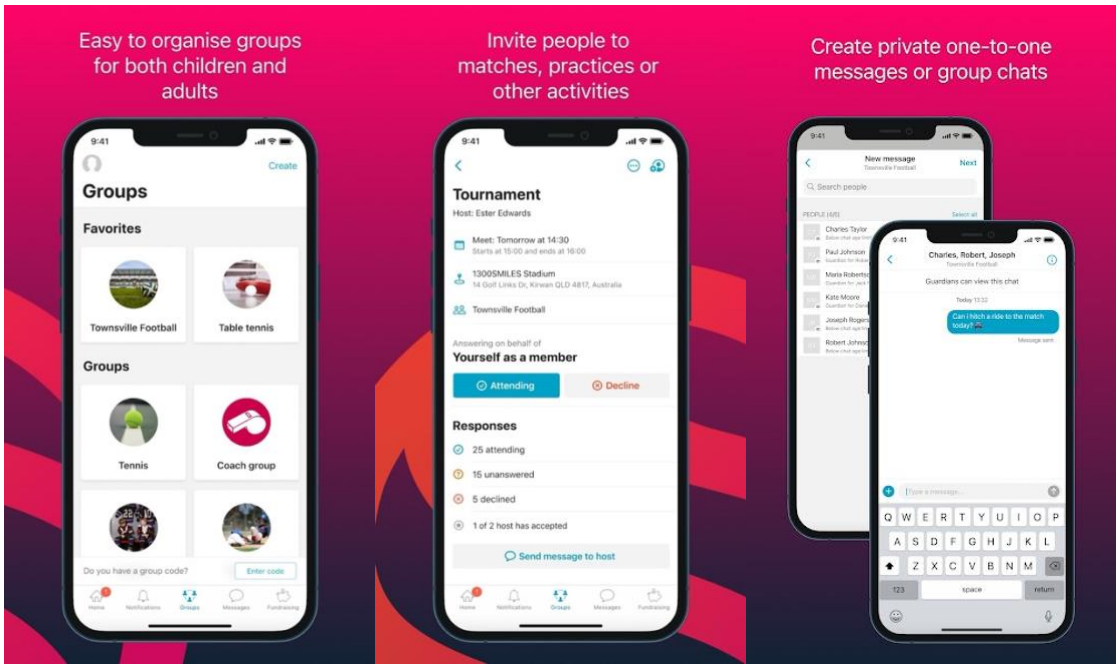


Figure 1 - Google Play Spond Screenshots (Spond AS)

Primary Features:

- Smart scheduling and event management with detailed RSVP tracking.
- Integrated payments collection for fees, subs, and kit costs via Stripe.
- Secure messaging with GDPR compliance and parental oversight for youth teams.
- Club-level administration tools (via their extension app; Spond Club)
- File storage for documents and safety information.

Availability: iOS, Android, and Web application.

Price: The core application is 100% free for teams and coaches. Spond generates revenue via fees charged on integrated payment collections.

While it serves as a high-standard baseline for scheduling and GDPR-compliant messaging, its functional scope remains focused on logistics. It does not provide any intelligent, tactical decision support.

Furthermore, its attendance and RSVP features provide data on availability, but Spond does not include performance analysis, predictive modelling, or player wellness tracking which might be important features for coaching staff for improvement. Its comprehensive administration features define the baseline functionality any competitor must meet, but its functional scope does not cover any real tactical and coaching support.

2.2.2 The Coaching Manual

One of the Coaching Manual (TCM)'s focal points is providing tactical information to enhance the way football training is planned and executed. They provide content to educate the football coaches with their target user being coaching staff and directors of clubs. By providing a wide range of software tools on their primarily website platform for training creation, they look to reduce the difficulty coaches endure whilst creating the team's training session, whilst also providing quality drills to better players. All TCM's coaching content is 'filmed with Southampton F.C in broadcast camera quality' (The Coaching Manual, 2023) highlighting a high level of standard. Their main features, besides their library of footballing knowledge, is locked

behind a paywall, where users must have the upgraded pro version. The actual application is stated to be used by multiple teams in different countries like Northern Ireland, the United States and Australia.

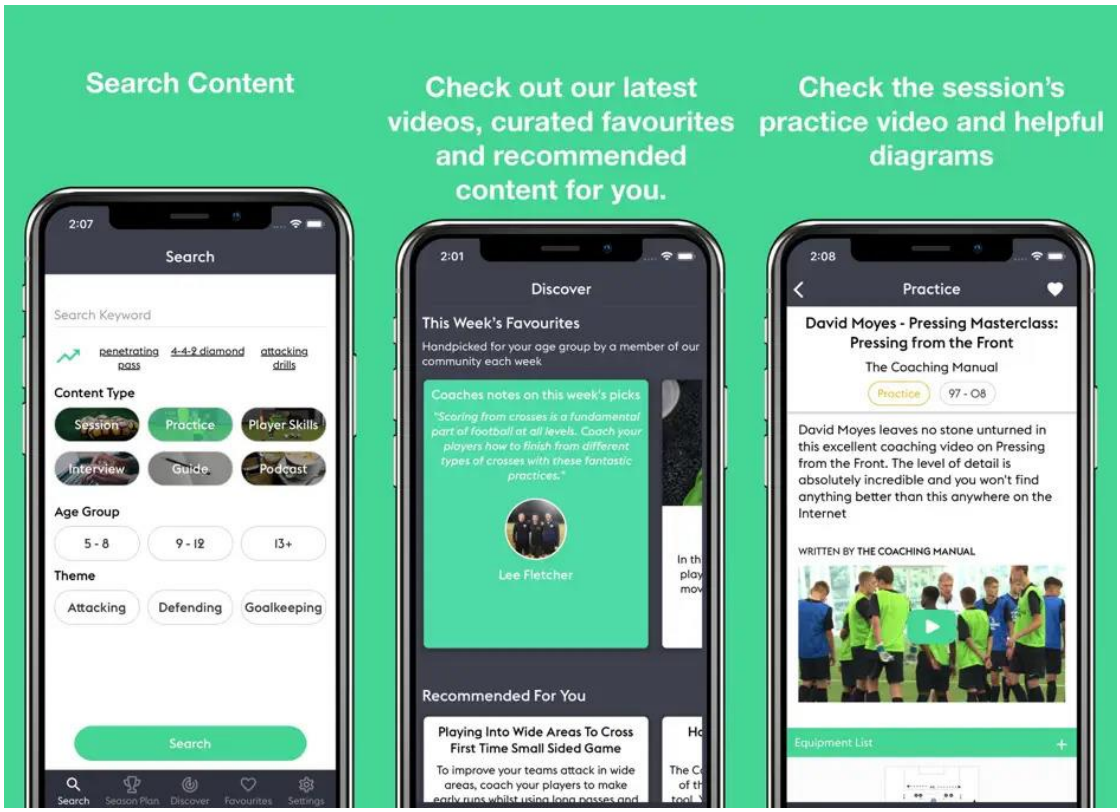


Figure 2 - The Coaching Manual App Screenshots

Features:

- Access to a library of 'professional standard' guides and interviews
- Access to an extensive list football coaching drills which can be filtered by ages and type
- Diagram creator – Coaches can create their own drill
- Practice creator – A tool where you can embed the created diagrams to a full training session plan
- Folders – a storage solution to add both curated preexisting library content or include created content.
- Team creation and member addition – Coaches and players with their guardians can be added to a team to view their upcoming training sessions plan.
- Players and parents can watch the upcoming session to gain a previous understanding.

Availability: Primarily web application. iOS and Android applications available.

Price: Access to all the services for every coach at the club for £140 a month

TCM excel at all features relating to football education for coaches. As they reduce the complexity of organising a session by providing a curated library of already created. From multiple user reviews, there is recurring issue brought up. There has been noted an inability to access the platform, putting off new user's from even attempting to try the application, with more than 5 user's leaving similar reviews in the span of a couple months. The application adds a time expense to coaches, but this could be beneficial in the long term. By learning more on all of football, coaches would be able to provide higher level training sessions and create

better strategies to suit their team. However, all the benefits depend on each individual coach's diligence to continue to gain football knowledge and their execution on gained knowledge.

A very large drawback is that it almost unusable without any payment. Thus, it is not easily accessible for grassroots team on tighter finances.

2.2.3 PlayerPulse

Originally known as SoccerPulse, the core application functionality focuses heavily on monitoring player health, load, and performance. Their apps provide a player interface for self-reporting on energy levels and other factors. With this, SoccerPulse can provide reports on individual player wellness for coaches based on the reports. They also include a focus on using the interactive questionnaires to get other metrics like fatigue, soreness, sleep quality and overall energy levels. They use the player's reports to display an aggregate score of 'overall readiness'. Furthermore, they also include a workload and training intensity monitoring feature based on more player feedback, which is crucial for preventing overtraining. The coach can generate a report whereby placing values on their general skills such as (dribbling, positioning, stamina) leads to a holistic ability score. The different technical and physical skills are added together and averaged with biased weighting included which is based on their position. An injury report can also be created by the players, which will inform coaches and keep them updated.

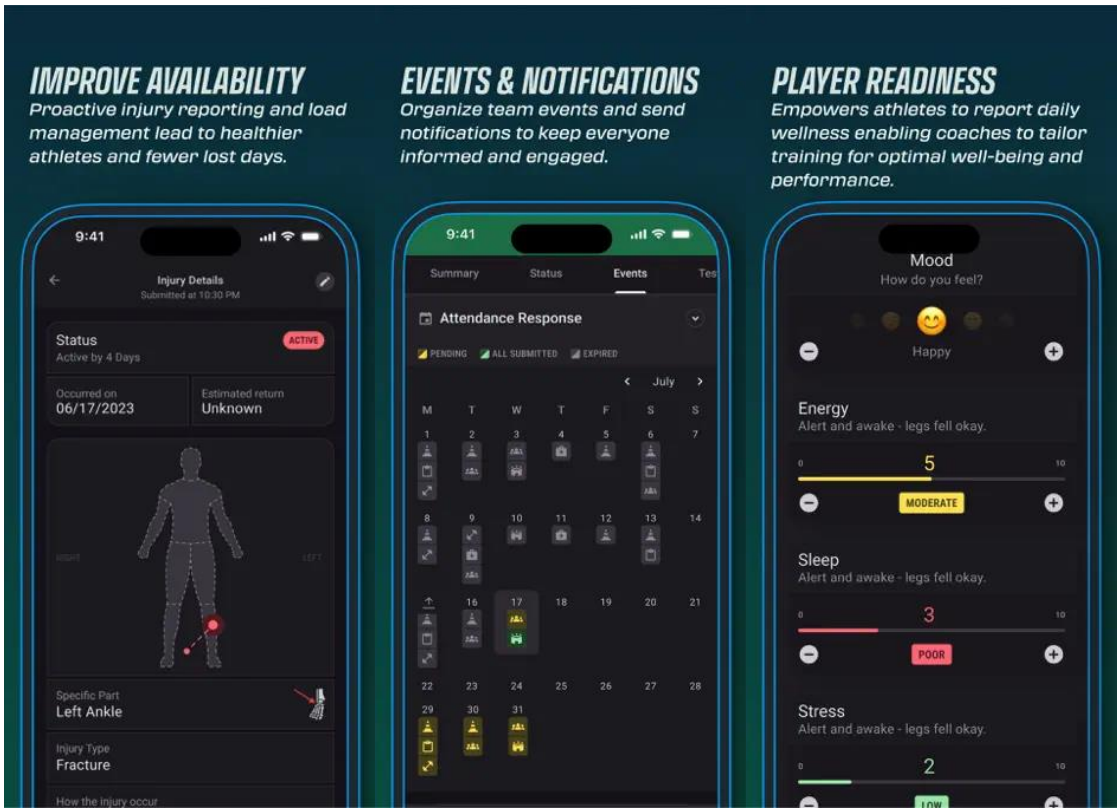


Figure 3- SoccerPulse Google Play Screenshots

Features

- Wellness reports – Athletes report on their readiness day to day.

- Injury reports – Athletes can log their injury updates, keeping coaches updated on recovery
- Performance evaluations – Both coaches and the players can submit scores of their training and game quality
- Dashboard – Display data of all types
- RPE and Training Load – Coaches can understand when to train teams harder.
- Event notifications – Coaches can organise team events and send notifications and reminders for training and matches
- Benchmark Recording – General sports fitness/strength results to track improvement

Availability: iOS, Android

Pricing : 36 dollars per year, per player.

PlayerPulse is an application designed for both players and coaches. By fulfilling their daily questionnaires, not only do the coaches gain an understanding of their player but they are able to have a steady stream of data to maintain their entire team. The application is a good application of using the primary necessary features; however the pricing does not make it suited for new amateur and smaller grassroots teams. Furthermore, they do not include any predictive modelling. The inclusion of their own messaging platform adds potential unneeded complexity to both coaches and players.

2.2.4 Pitchero

Designed for sports clubs, Pitchero is advertised as 'The complete club solution' providing a wide array of features. Pitchero provides the ability to create a club website with a free complimentary mobile application. They also include team management solutions, competition tracking alongside a robust. Furthermore, they also combine data from GPS trackers to provide training and match analysis. FIFA and World Rugby approve the PitcheroGPS Player Tracker for use at all levels of competition. Not much can be said on the quality of the analysis provided by these physical tools as details are not readily available.

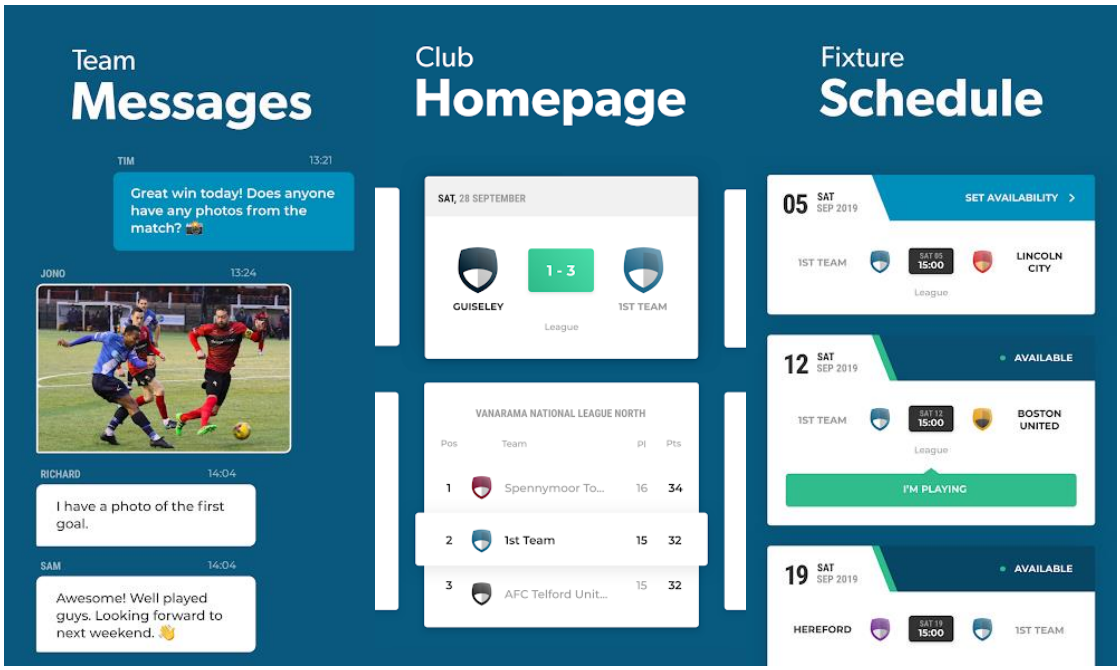


Figure 4 - Pitcheroo Google Play App Screenshots

Features:

- Player availability
- Team Messages
- Club homepage
- Fixture Scheduling
- Finance gathering support
- Club memberships

Availability: iOS, Android

Pricing: 36 dollars per year, per player.

2.2.5 Professional application in use

Kitman Labs l

2.2.6 Other applications worth mentioning

Heja - Its core value proposition is simplicity, aiming to get all players and parents onto a single communication platform, eliminating the need for fragmented SMS or private social messaging groups. Heja excels at administrative timesaving and team communication clarity. However, its focus like Spond is almost entirely organisational. It lacks any performance analytics tools, player wellness questionnaires, or advanced data synthesis. It does not address the core problem of providing intelligent decision support for tactical management, again like Spond.

SmartCoach++ – Somewhat like The Coaching Manual, SmartCoach offers a platform for building training sessions with their limited drill. However, as such a niche solution, SmartCoach++ does not offer such a wide selection of already made drills or videos to guide coaches on their coaching.

2.2.7 Comparison Table

Here is a table of all the applications research. The inspiration column refers to the key features which would fit in my application; however, it may not be feasible to add them as a requirement due to range of scope.

Solution	Notable features/advantages	Limitations	Inspiration
Spond	Available for parents; there is a focus on youth teams and the related safety properties. Large suite of features; from communication to finance management Good supporting documentation; easy to use	Heavily admin focused; Lack of performance analytics No football management support	GDPR Compliance Smart scheduling Overall intuitive UI design

The Coaching Manual	Rich training example videos library A feature for personalised training creation with good customisation.	Main features behind paywall Very narrow focus on football information provision No communication features	Training drill creator
SoccerPulse	Performance tracking Wellness Report Injury Report Training Load	Standard algorithms to process data; no predictive use Included messaging platform Very expensive	Repeated questionnaires Injury Report
Pitchero	External physical performance tracking using their own vests General solutions	Strong paywall	(Range of features are repeated, unfeasible and not needed.)

2.3 Research Gap

A critical evaluation of existing market solutions shows that applications typically specialise in either administrative efficiency or technical football education, but rarely integrate the two through intelligent automation. On one hand, platforms like Spond and Heja excel at the organisational side of grassroots management, significantly reducing the "administrative tax" through RSVP tracking and fee collection. However, these tools remain functionally limited to logistics and lack any meaningful support for tactical or coaching decisions.

Solutions such as The Coaching Manual provide high-quality technical football content but are often positioned behind significant paywalls and actually function primarily as digital libraries rather than coach assistants. While SoccerPulse attempts to bridge this gap by monitoring player wellness, its reliance on simple aggregate scores and standard algorithms lacks the predictive depth and automated planning capabilities that modern AI models can offer. SoccerPulse is also not accessible due to its price point.

The identified research gap is a lack of a focused, accessible application that prioritises the admin coaching side and purposefully uses the data a team can provide. With a small number of key features and the intelligent generation of predictive analytics, under-resourced amateur coaches can spend less ‘necessary’ time when trying to organise their team. Current "all-in-one" platforms often suffer from feature bloat, attempting to manage everything from finances to messaging, which can overwhelm student or volunteer coaches. By intentionally narrowing the project's scope to one or two core "technical" features, this study aims to address an underdeveloped sector of the grassroots game. This targeted approach ensures

that the application serves as a practical, data-driven assistant rather than a broad administrative dashboard.

2.4 Summary

The focus of the project is to face the challenge of meaningless time consumption in menial tasks, which in turn can alleviate the stressors on amateur coaches. Keeping coaches organised and ready is the priority. However, the amateur football coach has been identified trapped in a dilemma of administrative overload and financial deficit. The absence of tools to automate certain burdens represents a critical research gap. Without technological intervention to mitigate these operational difficulties, the sustainability of the grassroots game, and its associated benefits to public health and social cohesion, will be negatively impacted.

Chapter 3: 2Methodology

3.1 Methodology

For primary research to be done effectively, a practical structure must be set.

3.1.1 Research Onion

By systematically moving through each layer of the model below, the research methodology for this project will be constructed. By following the steps of the theoretical concept of Saunders' research onion, the primary research can be conducted with sensible direction. By moving from through each of the 'layers', each choice connects to the next and can support the final primary research's alignment with the research aims.

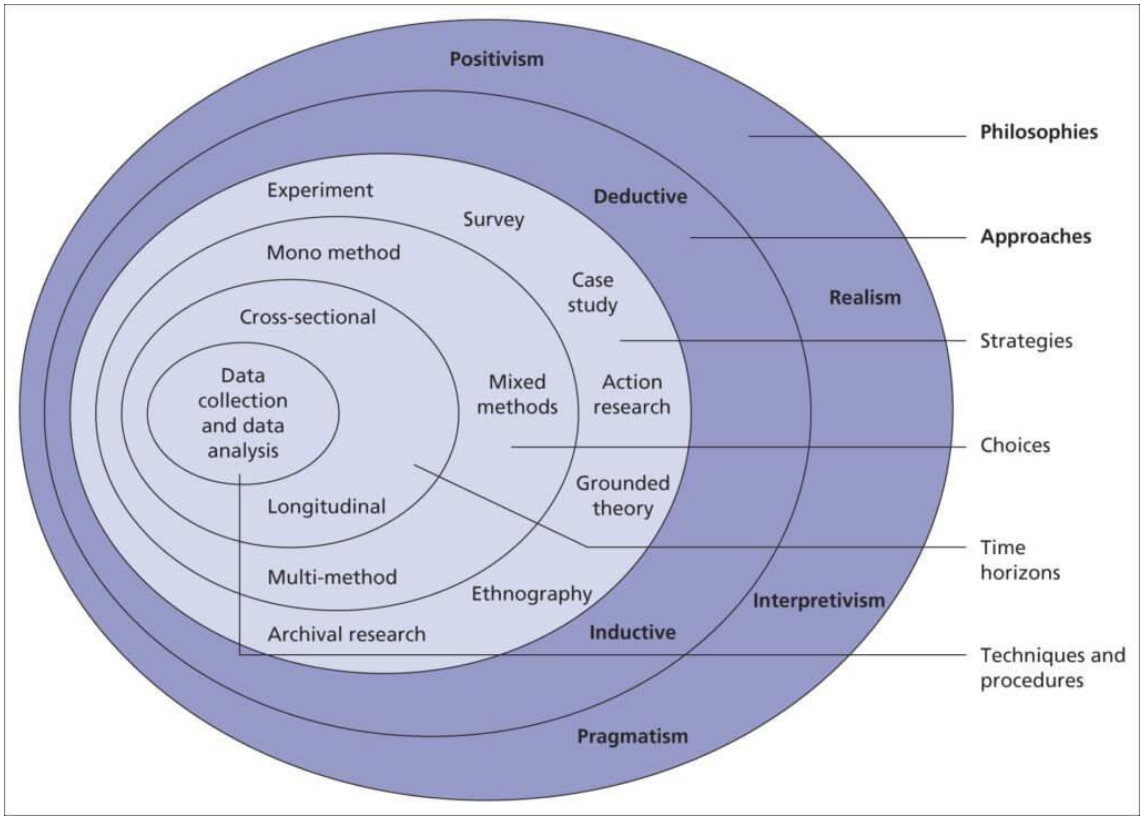


Figure 5. Research Onion (Melnikovas, 2018)

The outermost layer represents the research philosophies and refers to the set of principles the research is conducted on. Research philosophy can be described from either an ontological or epistemological point of view. This project will focus on the pragmatist philosophy. Pragmatism highlights the importance of using the best tools possible to investigate phenomena. The main aim of pragmatism is to approach research from a practical point of view, where knowledge is not fixed, but instead is constantly questioned and interpreted. For this reason, pragmatism is not committed to one specific philosophy and is considered flexible as it can use blend positivist and interpretivist positions. This adaptability will be useful as the focus is what works to solve the real-world problem of coach time resource scarcity.

The next layer is the approach. Between inductive and deductive, a deductive approach is suitable for this study. This approach means we can follow established theories in sports

informatics—such as the proven effectiveness of AI in forecasting player availability and workload in elite environments—and apply them to the resource-scarce amateur sector to test if they can be simplified to the level. To achieve this, this study will employ a mixed-method research design. This will involve two types of analysis: quantitative and and qualitative analysis. Quantitative analysis is used to gauge the technical metrics like workload ratios. Qualitative analysis is employed to gather data from the product users and see if the app is useful. These methods are executed through a design science research strategy, where the development of the app itself serves as the main tool for solving the problem.

Redo – approach layer does not reflect final app

Due to the academic timeframe, the time horizon for this project will be cross sectional, providing a snapshot of the system's real world impact at the end of the football season of 25/26.

The final layer of the research onion refers to the real practicalities of the research where choices of the specific techniques and procedures. The final layer of the onion involves practical techniques such as using the backend to process secondary industry data and primary player wellness inputs, ultimately transforming raw attendance and health logs into the predictive reliability and fatigue analytics required to support modern grassroots football management.

3.1.2 Initial Research and Requirements

After receiving approval from the ethics committee (if required by the university for using real-world coach/player data), primary research can be conducted. It will be focused on validation and requirements. This primary research will involve:

Surveys and Interviews: Administering a survey to grassroots football coaches (e.g., QMFC) to gather quantitative metrics on their current administrative time burden and qualitative insights into their specific decision-making processes when creating training

User Story Mapping: Using the collected user needs to define the features required for both the Coach Interface and the simplified Player Data Input Interface.

This research will allow the project to finalise the requirements and ensure the proposed application features directly address the identified operational pain points of the potential end-users.

3.2 Primary Research Gathering

3.2.1 Survey

The survey questionnaire is included in Appendix A.

The objective of the survey is to engage with the target user base to ascertain the appropriate core functionalities that are to be prioritised in the application. By understanding the potential users' specific coaching struggles and their desired features in an assisting app, the app's direction and priorities can be solidified. The data gathered will inform the basis of the project, with the response being used to highlight the functional and non-functional requirements lists ensuring the features focused would satisfy participants, whilst also filling the identified gaps of existing solutions.

3.2.2 Participant Recruitment

Participants were selected based on having previous experience in either voluntary or paid coaching roles with a focus on those working with adult groups of footballer. An application for approval from the ethics committee to conduct the survey was submitted with all the essential supporting documents needed.

3.3 Data Collection and Analysis

Per the methodology plan, the inner most is data collection. The method was questionnaires, which most were sent as online forms and emails, however some answers were recorded in person. I received over 20+ responses, with the majority coming as google forms.

The first questions were to gauge the general data of each football team. I learnt that most coaches had mainly 11-16 players on their team, with very few amateur teams having over 20+ players highlighting the lack of depth of a squad. In comparison, the average squad number of a NLS (Nation League System) club being 23 players. Most clubs have come into contact 2 times a week. The actual time spent on most of the admin is mainly reported to be 2-3 hours, with none of the applicants say it takes less than 2 hours.

The use of WhatsApp is very prominent in handling all of communication related aspect of the club. The specific applications taking a smaller number of the participants attention with under 30% being the result. It is clear throughout the responses that WhatsApp is to be a core application that most will continue to use so any features in tandem with it should be useful and not overcomplicate. It also highlights the simplicity that the app must have as previous apps included their own communication platform. Reponses indicate it would be easier to integrate what is already in use.

Performance insights and automated attendance tracking were reported as the most popular features that would be useful. This changes the requirements to focus on these features as primary targets of implementation.

In regards to the use of AI within a usable applications, I recognised a trend where older coaches are more opposed to AI, but the majority of people definitely are open to the use some sort AI or AI adjacent assistance, as long as it necessary. There was a particular comment which captured the overall sentiment of coaches during the extra comment sections, which reported their concern for losing some of the personal aspect of coaching with the use of technology. A focus on using “technology for the administrative tasks” and not the personal aspect of lineup selection as tech “doesn’t understand individual player complexities”.

Overall, the answers highlighted a familiarity with technology, and highlighted the need to accessibility within the application between different platforms, and general usage of the app must be as fluid and intuitive for the less tech savvy.

3.4 Risk Register

A risk register is to proactively helping the completion of the project by identifying potential disruptions or faults that could occur and block the achievement of key objectives. This register outlines the risk, its probability, the severity if it where to occur and the corresponding mitigation strategies that should be employed to secure successful completion.

ID	Risk	Probability	Impact	In event's occurrence?	Prevention Methods
1	Insufficient number of human participants for requirements research	High	High	The final application could not be useful for the primary target with a lack of clear direction provide from target audience feedback	Identifying potential participants early. Expanding the criteria to exceed numbers
2	The primary research done is not helpful	Low	High	The solution will not entail features that are useful in a real coaching environment	The questions chosen were curated and deliberated with purpose and consideration
3	Solution fails to meet the users requirements	Low	High	The application will not align with the user's wishes leading to an unused app.	Focusing on completing the user requirements as essential parts of the project.
4	API integration issues	Medium	Medium	Any use of external messaging applications will not occur, cutting off a large feature	Reading documentation and ensuring correct API are used.
5	Unpredictable events / illness	Low	Medium	A delay in the total completion of the project leading to potential misses on key features and functionality	Priorities core deliverables and attempt to create as much buffer time i.e stay ahead.
6	Final solution has unmet requirements	Low	High	The features are not useful for the user base.	Again, focus on requirements is essential.
7	Incorrect AI predictive abilities	Medium	Medium	The predictions created by the Artificial Intelligence are unusable	Reduce the complexity of the prediction if needed, but an accurate and helpful AI addition is

					essential and targeted.

Chapter 4: Requirements and Design

After receiving the green light from the ethics committee, the project's primary data collection phase can begin. All the collated results will feed into the creation of the final application

4.1 MoSCow

Functional requirements refer primarily to features that the project must implement to enable the users to achieve their goals. Non-functional requirements refer to the project attributes which are more 'abstract'. The non-functional requirements impose constraints on the implementation of the functional requirements in terms of performance, security, reliability, scalability, portability, and so on. These requirements will make up this project's main targets to determine how well we executed this project. The requirements will follow the MoSCow prioritisation categories, as this system creates a distinct category based on the requirement's significance. The division is into 4 categories: 'Must have', 'Should have', 'Could have' and 'Will not have'. 'Must haves' are invaluable for the overall project success and so their inclusion in the project is mandatory. 'Should have' and 'could have' are similar in terms of their role as secondary priorities. 'Should have' requirements are important features that are not strictly vital for the system to function but add significant value and should be included if time and resources permit. 'Could have' requirements are desirable features that would be nicer to have but have a smaller impact on the core objective and can be easily omitted if the project timeline becomes pressured. Finally, 'Will not' have requirements are features explicitly excluded from the current project scope, helping to prevent "scope creep" and ensuring focus remains on the primary deliverables.

Can be found at (Product Plan, 2025)

4.2 Requirements

An initial set of requirements were created whilst compiling the primary research results. As all the primary data was finalised, the analysis of the data adjusts the urgency of requirements creating a new final set of requirements. These 'pre-' and 'post-' survey set of requirements were added at Appendix B with the changes reasoned according to the gained understanding of primary research.

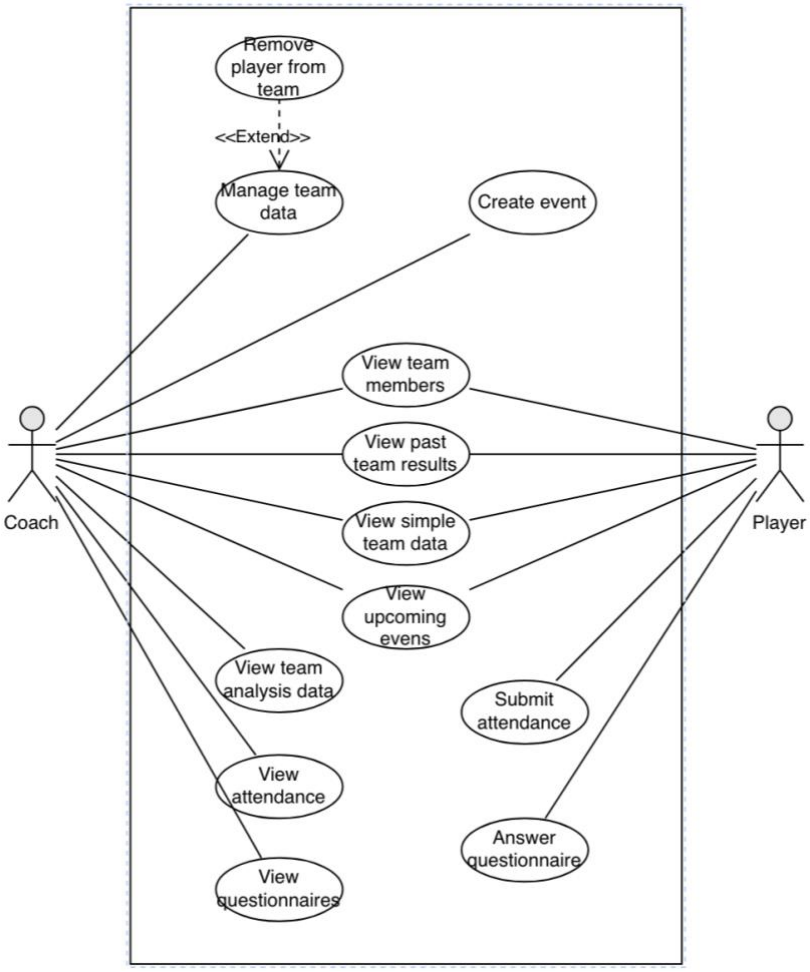
The requirements change highlighted the importance of …

4.3 Design

With the requirements finished, we can begin creating

4.3.1 Use case diagram

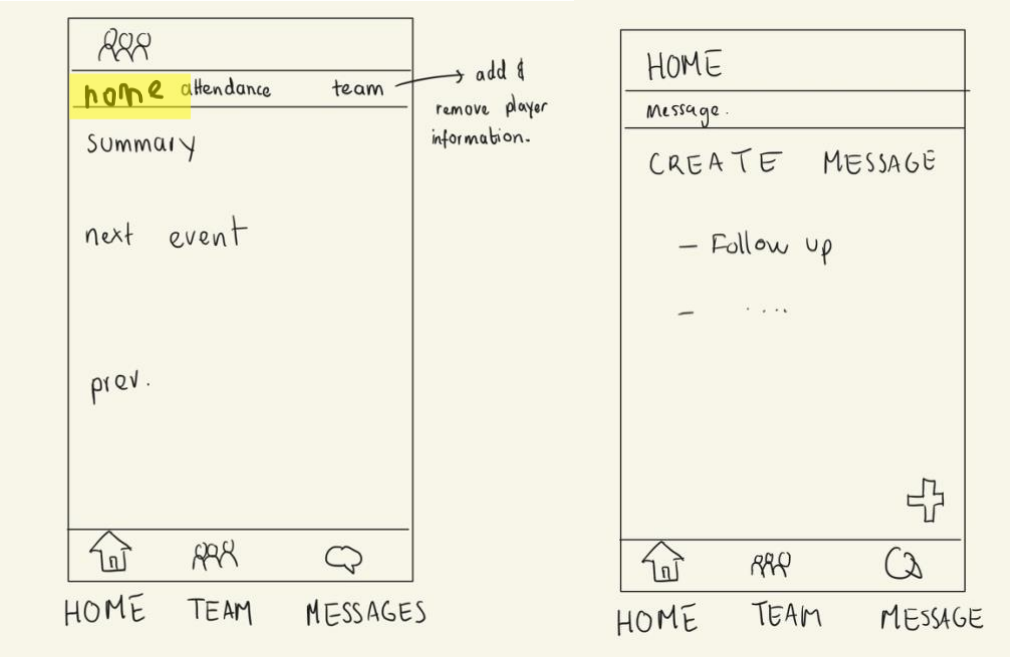
The figure below describes the applications use case diagram for the types of authenticated users. The system has 2 main actors – Player and Coach, which can login after they have created their account. There are split functionalities as the Player requires only a simplified interface to answer questionnaires which will support their Coach's management.

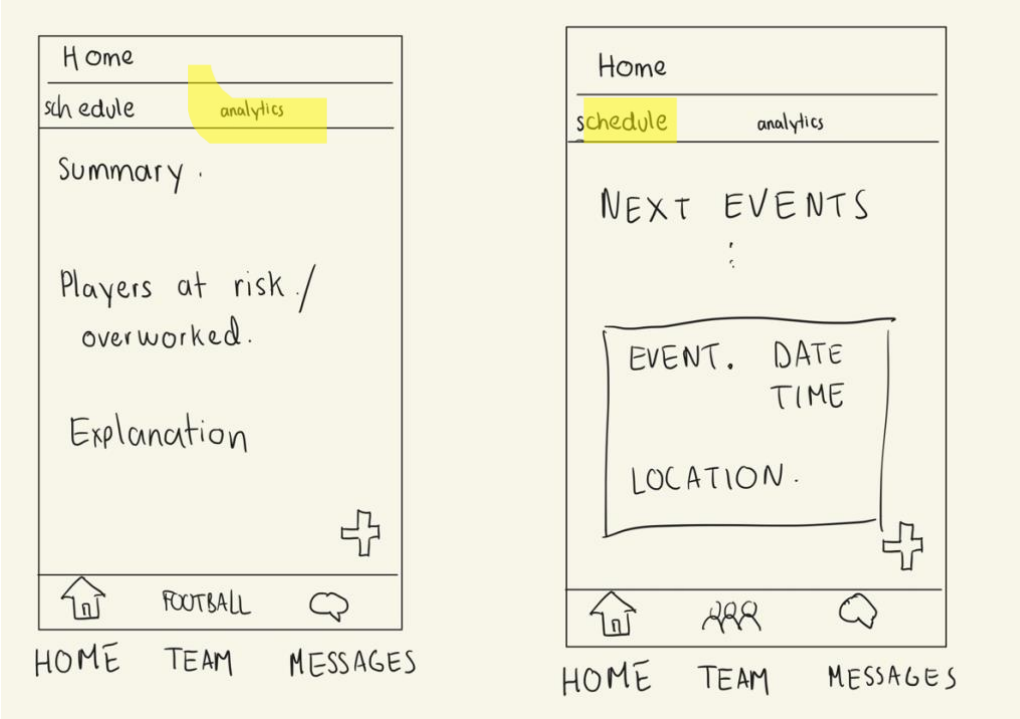


4.3.2 User interface

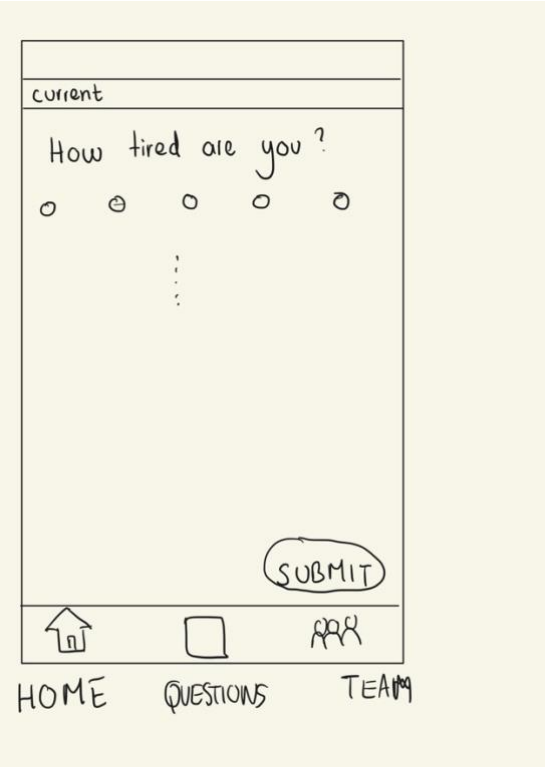
The user interface is a fundamental component of the design as it ensuresl that the results of complex data processing of AI models can be displayed in a digestible manner for our non-technical users. For a tool aimed at amateur and grassroots level coaches, an intuitive UI is needed to reduce the volunteer time spent on organisation and encourage data entry. Due to the nature of manual data entry, a fluid and responsive UI will also support the completion of the questionnaires as a negative application experience will quickly demotivate players completing them. The two figures below show the mock-up for the Coach interface at its home dashboard view and then its most important pages, the schedule page, questionnaire page, and messages page. From the home screen, coaches can view a high-level summary of their team profile and recent training attendance logs. Prominent buttons allows coaches to access the different aspects of the application with ease.

To address the core objective of intelligent decision support, the analytics page is a button at thte top of the landing page (the home page), where the results of our model can predict the fatigue and warn of overtraining or lack of players.





Below, the figure illustrates the Player version of the interface. This simplified view focuses on ease of data entry to assist the coach in gathering accurate metrics. Players can quickly log their wellness and overall health questionnaire. At the bottom of both interfaces, a navigation bar enables users to switch between home view, team and questionnaire. Home only shows their schedule whilst team only show basic information of their other teammates.



4.3.3 Technological stack

For the development environment, Visual Studio Code was chosen due to its many plugins and preestablished familiarity. Django was considered for the backend of this stack, however due to the use of AI, FastAPI is better suited due to its native support of asyncio, which allows a much easier use of external API calls within in the program as it allows the server to handle other tasks while waiting for a response. FastAPI also is more lightweight. React Native is going to be used due to the seamless cross platform development. Our target users use a mix of Android and iPhones and this is a strong requirement for accessibility. Firebase is the choice to finish the applications' ecosystem, as it combines well with both frontend and backend. Firebase also offers offline persistence through native data caching which is also a key requirement.

4.4 SLDC

The development of this project will follow the Software Development Life Cycle (SDLC) to establish a structured roadmap from initial requirements gathering to final delivery. An agile approach will be adopted for the majority of the implementation, as its iterative nature allows for constant refinement based on stakeholder feedback. By delivering small, functional increments, I can present prototypes to users early and incorporate their insights into the next development cycle, ensuring the final product aligns closely with their actual needs.

To maintain code integrity and security, the project will be hosted in a private repository on GitHub. This makes sure the codebase is not tied to a single physical machine and facilitates easy collaboration or review if external access is required in the future. Also, using version control allows for a clean history of changes, making it straightforward to revert to previous stable versions if technical issues arise during development.

For the deployment phase, the software will be packaged as a standalone executable file. This method is chosen for its simplicity, as it allows any user to run the application immediately without needing to install specific dependencies or have a technical understanding of the underlying code. However, a notable constraint of this approach is that it does not support remote "push" updates. Consequently, any future bug fixes or feature enhancements will require the user to manually download and install a new version of the executable to replace the existing one.

Chapter 5: Implementation

5.1 Development plan

The following is a week-by-week plan to structure what needs to be done including features and regular framework. Testing is to be conducted throughout each week.

- **Week 1: Working on the core infrastructure.**
  A basic frontend will be set up which is to consist of react native navigation of pages, alongside the basic authentication for all users. Database connection will be setup.

- **Week 2: Functional logic and the player experience.**
  Build the logic for coaches to create events and for players to log their wellness via a simplified interface.

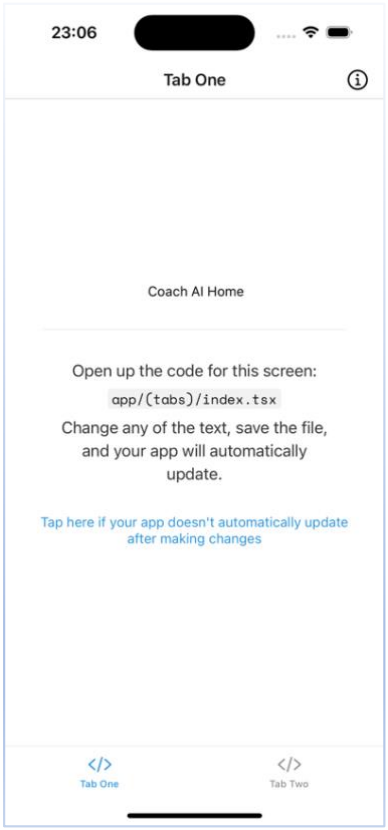
- **Week 3: AI predictive modelling and analytics**
  Develop the primary technical aspect of using the data, using the data gathered from the data inputs.

- **Week 4: Analysis dashboard and extra features**
  Adding all the dashboard,

- **Week 5: Refinement and final testing**
  Refine the UI to ensure the non-technical coaches can navigate easily. Package the backend and the frontend into their final deliverable.

5.2 Project start

5.2.1 Setting up the environment



Git is used for the tracking. Github was used to store the main codebase. The project was worked on a local directory using a Macbook Air and then eventually a Macbook Pro. Conda was used for the python frontend packages management and nvm was used to manage the system's npm version. All these package management tools were used to make the build process easier to produce and redistribute. Furthermore, the ease of management sped up development.The project's frontend base started with a React Native app skeleton using Expo. The tabs template was used to speed up the development as it aligned with what I wanted to make. Using Expo's tutorial, there was an initial template of our application which is shown below (Figure 3).

5.2.2 Initial start complications and boilerplate

During the setup, there were errors of the version of node which was on the development environment. The easiest solution was to delete all files belonging to node and its other installed packages, and then installing it again using expo's built in package.

Complications arose after trying to install NativeWind alongside Expo. By trying to also load the web server, this test revealed there could be a problem with the loader.

```
iOS node_modules/expo-router/entry.js                    99.9% (1574/1575)
Web node_modules/expo-router/entry.js                    98.7% (1201/1209)
λ node_modules/expo/node_modules/@expo/cli/node_modules/@expo/router-server/node
/render.js                                               99.8% (1173/1174)
```

I eventually found that the problem were inside my global.css file. Following the tutorial incorrectly lead to

5.2.3 Bundling React Native Issues

During the development of the infrastructure, a problem occurred during the bundle phase of the React Native application.

There was a Babel validation error which stated that ".plugins is not a valid Plugin property." This conflict was created by the mismatch between nativewind v4 and the expo router's entry point. Babel encountered a recursive nesting error by incorrectly identifying a configuration

object as a plugin containing its own plugins property. With the aid youTube, the problem was resolved this by changing the Babel configuration to move NativeWind from a standard plugin to a high-level preset while simultaneously setting the jsxImportSource to nativewind. This made sure the styling engine was prioritised before the animation transformations.

A secondary Exception in HostFunction was addressed by aligning the react-native-reanimated and react-native-worklets versions using the “npx expo install –fix” command to make sure they were synced. This stabilised the system and allowed a solid base render for the primary page on the iOS simulator.

5.2.4 Initial Tab Work and Supabase Start

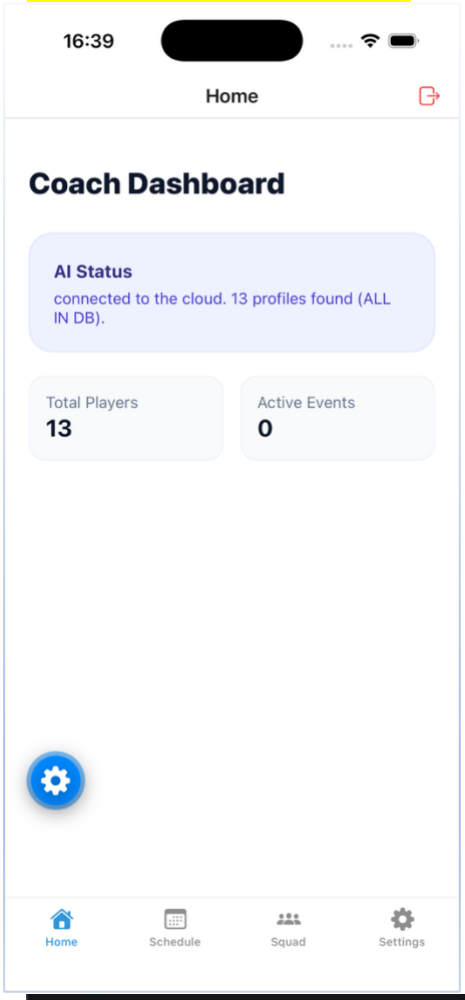
All tabs were modified to fit our brand and added custom text to test navigation. After implementing it, my focus shifted to integrating Supabase to handle user authentication and user data storage. Firstly, I executed the SQL command on the Supabase web side to create our Supabase database tables, online

Initial SQL Team Management Schema

```
-- 1. Profiles Table (Both the coaches and players)
CREATE TABLE profiles (
  id UUID REFERENCES auth.users ON DELETE CASCADE PRIMARY KEY,
  full_name TEXT,
  role TEXT CHECK (role IN ('coach', 'player')),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  whatsapp_number TEXT
);

-- 2. Teams Table
CREATE TABLE teams (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  team_name TEXT NOT NULL,
  coach_id UUID REFERENCES profiles(id),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  join_code TEXT unique
);

-- 3. Events Table (Matches,training or other)
CREATE TABLE events (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  team_id UUID REFERENCES teams(id),
  event_type TEXT CHECK (event_type IN ('training', 'match')),
  event_date TIMESTAMP WITH TIME ZONE,
  location TEXT
);
```

```
-- 4. Attendance & Wellness Logs
CREATE TABLE attendance_logs (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  event_id UUID REFERENCES events(id),
  player_id UUID REFERENCES profiles(id),
  status TEXT CHECK (status IN ('attended', 'absent', 'late')),
  fatigue_score INTEGER, -- For your AI predictive modeling
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- 5. Questionnaire Table
CREATE TABLE questionnaires (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  player_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
  event_id UUID REFERENCES events(id) ON DELETE CASCADE,

  -- Quantitative Data (for AI/Maths analysis)
  team_performance_rating INTEGER CHECK (team_performance_rating BETWEEN 0 AND 10),
  tiredness_level INTEGER CHECK (tiredness_level BETWEEN 0 AND 10),
  player_performance_satisfaction INTEGER CHECK (player_performance_satisfaction BETWEEN 0 AND
10),

  -- Qualitative Data (for Coach insight)
  suggestions TEXT,

  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

  -- Guardrail: Ensure one player only submits one questionnaire per event
  UNIQUE(player_id, event_id)
);

CREATE TABLE lineups (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  event_id UUID REFERENCES events(id) ON DELETE CASCADE,
  formation_type TEXT DEFAULT '4-4-2', -- The "Key" for your frontend hard-coded shapes
  player_positions JSONB, -- Stores { "GK": "player_uuid", "RB": "player_uuid", ... }
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

Explain the use of SQL a little bit



I added ratata

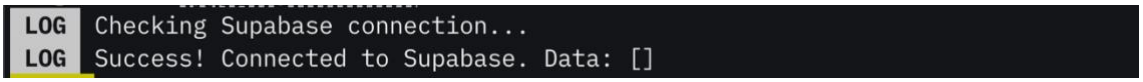
I tested the connection by adding a simple Supabase display message, which would show how many users were in the database, and an initial connection success message . Here is where I made fake users to check for the app's constant connection, which was successful. The number of users matched up with

5.2.5 Bridging the frontend and database

After installing the necessary dependencies for the supabase connection, I ran into some dependency clashing.

```
warning: Bundler cache is empty, rebuilding (this may take a minute)
The following packages should be updated for best compatibility with the install
ed expo version:
  react-native-get-random-values@2.0.0 - expected version: ~1.11.0
  react-native-reanimated@4.3.0 - expected version: 4.2.1
  react-native-worklets@0.8.0 - expected version: 0.7.2
Your project may not work correctly until you install the expected versions of t
he packages.
iOS Bundled 58285ms node_modules/expo-router/entry.js (1637 modules)
WARN Route "./_layout.tsx" is missing the required default export. Ensure a Re
act component is exported as default.
ERROR [Error: Exception in HostFunction: <unknown>]

Code: _layout.tsx
> 1 | import 'react-native-reanimated';
    | ^
  2 | import "../global.css";
  3 |
  4 | import { DarkTheme, DefaultTheme, ThemeProvider } from '@react-navigation/
native';
Call Stack
  <global> (app/_layout.tsx:1)
```

Regardless, after creating a check, we found that by using the older method of connection, there was indeed a connection between our database and the react native. Which means we can now move onto the main part of our code and design our website.



5.2.6 Tab setup

The original tabs template only provides us with 2 tabs, whereas 4 tabs are necessary. As the native template we have chosen has already organised the project as a file-based page system, we just added new files under our tabs directory. The index file is our only unchanged file as the app needs a point to begin whose name must not be anything but tabs.

5.3 User authentication

Supabase has its own separate authentication system which can be used out of the box and then synced with our own database using hooks. So we could either create our own authentication and deal with all of the double checking complexities and atomic properties or leverage the tools provided by Supabase.

5.3.1 Root Level Protected Routing

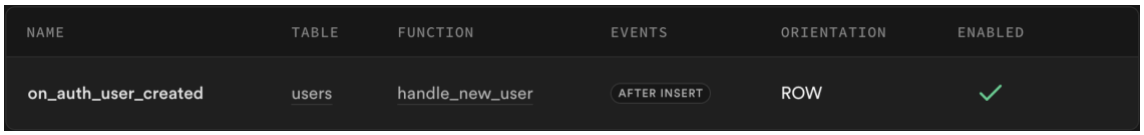
The application structure was refactored to incorporate a protected routing system. Only authenticated users are allowed to access any of the app, so I added this guard rail to push all non-logged users out to sign in screen. This was achieved by implementing a root-level session listener that interfaces with the Supabase authentication engine upon application launch. By utilising the useSegments and useRouter hooks, the system monitors the user's current directory within the file structure. If no active session is found while a user attempts to access private directories such as the team or schedule tabs, an automatic redirect to the login screen is triggered. This method provides a strong security layer by preventing private screens from being initialised in memory. Centralising this logic within the main _layout.tsx file creates a single point of control for application security, which improves the management of the codebase as the project grows.

5.3.2 User onboarding

I created the onboarding subsystem using the Supabase GoTrue library to manage backend user accounts and session persistence. The sign-up interface was designed to collect user credentials while also capturing additional metadata, such as the full name and the assigned system role. This integration allows for the storage of coach attributes directly within the authentication object, eliminating the immediate requirement for secondary database queries during the initial setup. Security standards were maintained through the use of secureTextEntry for password inputs and autoCapitalize="none" for email fields to reduce user error during the authentication process. Once a session is successfully verified, the root layout transitions the user to the main dashboard area. This architecture makes sure that the identity of the coach is securely linked to all associated player data and scheduled events.Team screen.

The most important part was syncing our authentication layer and the application data. In the initial development phase, the creation of a user account in the auth.users schema did not automatically generate a corresponding record in my public.profiles table. To resolve this, a PostgreSQL trigger within the Supabase environment function was authored to intercept new

identity registrations and perform an atomic insert into the relational database. The solution extracts the coach's full name and role provided during the sign-up process. The reason I implemented it this way as client-side insertion adds the risk of "orphan" accounts (where a user exists but has no profile) caused by possible network interruptions or application crashes that may occur during the onboarding sequence. The Supabase trigger is shown below.

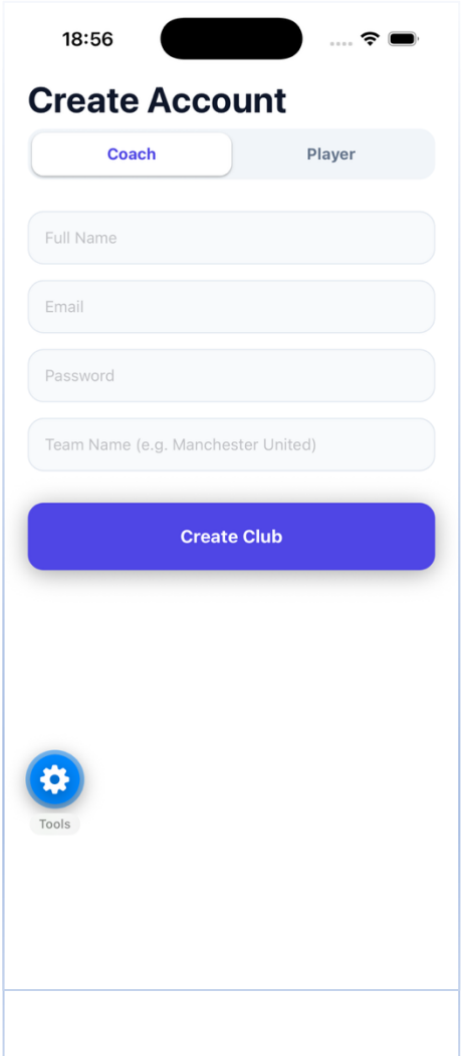


Trigger

```
BEGIN
  INSERT INTO public.profiles (id, full_name, role, team_id)
  VALUES (
    new.id,
    new.raw_user_meta_data->>'full_name',
    new.raw_user_meta_data->>'role',
    (new.raw_user_meta_data->>'team_id')::uuid
  );

  -- If the user is a coach, update the teams table to link this coach_id
  IF (new.raw_user_meta_data->>'role' = 'coach') THEN
    UPDATE public.teams
    SET coach_id = new.id
    WHERE id = (new.raw_user_meta_data->>'team_id')::uuid;
  END IF;

  RETURN new;
END;
```

5.3.3 Improving sign in pages

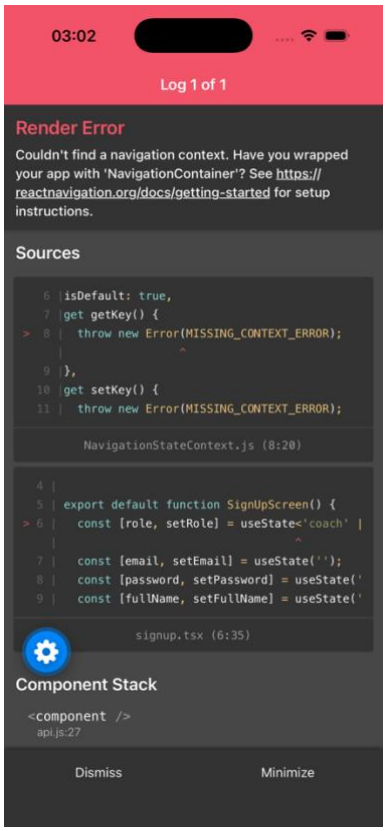
Following traditional app flows, The “Do not have an account?” callout navigated from login to sign in pages and the “Already have an account?” button did the opposite as expected. There is a difference in data input from a



During the integration of the role-based onboarding flow, a transactional error was identified that prevented the successful registration of new users. Investigation revealed that the failure originated within the PostgreSQL trigger function, specifically during the type conversion of metadata strings into UUID format. To resolve this, explicit type casting was implemented to ensure that team identifiers provided by players were correctly parsed by the database kernel. Additionally, null-handling logic was added to the coach provisioning sequence to prevent system crashes in instances where optional metadata fields were absent such as phone number and etc. These refinements improved the resilience of the authentication pipeline, ensuring that the database can gracefully handle variations in user input while maintaining strict relational constraints between the identity and profile layers.

To facilitate a secure multi-tenant environment, the squad management module was updated to utilise dynamic relational filtering. Rather than retrieving all records from the profiles table, the application executes a hierarchical query. The process initiates by identifying the authenticated user's unique identifier (UUID), which is then used to retrieve the corresponding team_id from the profiles table. Subsequently, a secondary query is performed to isolate only those records where the team_id matches the coach's affiliation and the role is designated as "player." This ensures that the user interface exclusively displays data relevant to the coach's specific organisation. Furthermore, alphabetical sorting was applied to the resulting dataset to enhance the usability of the roster management system.

To facilitate efficient session control, a global logout mechanism was integrated into the application's primary navigation header. By utilising the screenOptions attribute of the Tabs component, a persistent trigger was established across the Home, Team, and Messages modules. This implementation utilizes the supabase.auth.signOut method, which, upon invocation, invalidates the local session and triggers the root-level authentication guard to redirect the user to the login directory. The interface employs a standardised exit icon with a distinct red tint to differentiate account actions from operational coaching features. This design ensures that session management is consistently accessible, improving the security and usability of the mobile environment.



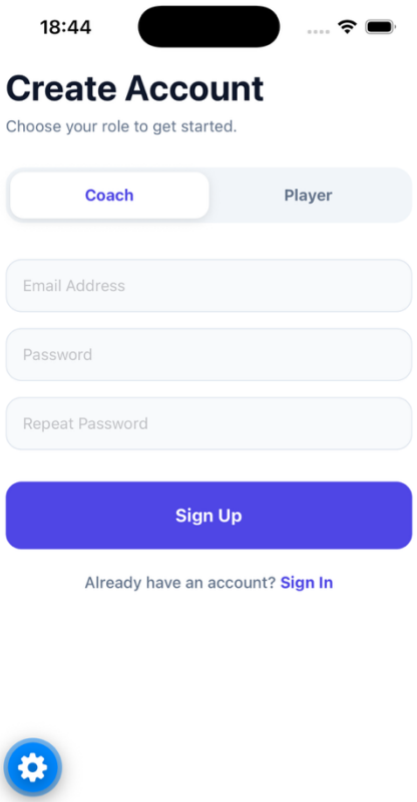
5.3.4 Navigation Provider Hierarchy Error

The sign in page

In a React Native app, the navigation provider occurs when a component is not ‘in range’. By attempting to implement this toggle feature using a native modal, the app created a separate leaf outside the main navigation tree. When the toggle button was clicked, the component tried to communicate with the main navigation provider but could not and thus the navigation context error screen appeared.

Rewrite this

The Core Issue: Navi
gation Provider HierarchyIn a React Native app, the Navigation Provider is like a radio tower. It broadcasts navigation signals to every screen. The "Missing Navigation Context" error happens when a component is "out of range" of that tower. In your case, because you were using a native Modal, the app created a separate window that was physically outside the main navigation tree. When you clicked the toggle, the component tried to "talk" to the tower, couldn't find it, and crashed the app. 2. History of Fix AttemptsTo solve this, we moved through several layers of the application to isolate the failure point. Attempt 1: Prop AlignmentAction: Standardised the function names between the parent screen and the modal (changing onSave to onEventCreated). Result: Fixed the "undefined function" crash but didn't solve the underlying navigation context issue. Attempt 2: Library Conflict ResolutionAction: Ensured all components like TouchableOpacity were imported from react-native instead of react-native-gesture-handler. Result: Reduced library "noise," as some libraries require extra wrappers (like GestureHandlerRootView) that break inside modals. Attempt 3: Route-Based Modal (The Architecture Shift)Action: Deleted the popup component and turned "Add Event" into a real page (app/add-event.tsx) registered in the root _layout.tsx. Result: This brought the screen "back in range" of the navigation tower, though the toggle still crashed during re-renders. Attempt 4: State Memoisation (useCallback)Action: Wrapped the toggle logic in useCallback to prevent the function from being "re-born" every time the screen updated. Result: Stabilised the logic, but the styling engine was still causing a context check. 3. The Final Fix: Pure Style IsolationThe definitive solution involved two mechanical changes:Removing Styling Engine Conflicts: We replaced NativeWind (className) with standard Inline Styles for the toggle. This prevented the styling compiler from triggering a "layout measurement" that was accidentally asking the navigation tower for data mid-click. Using Pressable: We swapped TouchableOpacity for Pressable. Pressable is a "dumb" component that doesn't care about navigation, making it much safer to use in complex layouts.



5.3.5 Reducing user onboarding friction

From a UX standpoint, adding too many fields to the initial account creation would overwhelm any user, so by just including the minimum required fields for a user to be created, we can split the other less important field entry after logging in into another screen. This facilitates a frictionless setup and adds a role-specific experience as a team player only requires their name when creating an account. However, for a new coach, creating a team name is essential.

To make sure the new onboarding screen is a one-time popup we add a check, pulling a profile field called ‘onboarding_done’.

5.4 Schedule

Fetching the Supabase variables

Keeping the RLS policies in mind, we

```
const fetchEvents = async () => {
  setLoading(true);
  try {
    const { data: { user } } = await supabase.auth.getUser();
    if (!user) return;

    const { data: profile } = await supabase
      .from('profiles')
      .select('team_id, role')
      .eq('id', user.id)
      .maybeSingle();

    if (profile) {
      setUserRole(profile.role);
      if (profile.team_id) {
        const { data, error } = await supabase
          .from('events')
          .select('*')
          .eq('team_id', profile.team_id)
          .order('event_date', { ascending: true });

        if (!error) setEvents(data || []);
      }
    }
  } catch (err) {
    console.log("Fetch error:", err);
  } finally {
    setLoading(false);
  }
};
```

5.5 Availability

5.6 Questionnaires

5.6.1 Creating a questionnaire as a coach

Prioritising user experience is a high priority so executing the questionnaire section as close as possible to. In earlier designs, users were allowed to customise the questions inside the questionnaire given to players, however due to time constraints, the feature was limited to a standard set.

5.6.2 Answering a questionnaire as a player

5.6.3 Viewing questionnaires

Since questionnaire are another aspect which is individual to each event, there was a button toggle

5.6.4 Connecting backend with database

```python
# Check if keys actually loaded to avoid type errors
if not url or not key:
    raise ValueError("Check your .env file, the keys didn't load.")

supabase = create_client(url, key)

def run_test():
    try:
        # just pulling one row from teams to see if the door is open
        response = supabase.table("teams").select("*").limit(1).execute()

        print("Connection Successful")
        print("Data retrieved:", response.data)

    except Exception as e:
        print("Connection Failed")
        print("Error details:", e)

if __name__ == "__main__":
    run_test()
```

5.6.5 Converting into AI insight

To ensure the statistical validity of the AI-generated squad reports, a threshold-based activation logic was implemented. The system is configured to suppress Generative AI (GenAI) calls until a minimum quorum of five player questionnaires is met for a specific event. This "Threshold Guard" prevents the AI from generating skewed or non-representative advice based on isolated data points. Furthermore, a version-control mechanism was established within the ai_insights table, where each stored insight is tagged with the response_count at the time of its creation. The Python middleware only retriggess the LLM synthesis if the current

questionnaire count goes above the recorded count, which in turn optimises the API token usage and ensures that the coach is always presented with the freshest overview.

5.6.6 Resolving Exception States in Python Middleware

5.7 Home or Analysis Page

5.8 Summary

m

Chapter 6: Testing

Testing is important to gauge what parts of the application function as purposed. Among the plethora of testing methodologies, unit tests, component tests, and integration tests stand out as fundamental practices. Each have different characteristics which make the suitable for different tasks. Since our application is a harder to break down into units as most of it is a combination of technologies, we will not be including unit tests.

Integration tests evaluate the interaction and integration of multiple modules or subsystems to ensure that they function correctly when combined. Unlike unit tests and component tests, which focus on isolated parts of the system, integration tests validate the behaviour of the system as a whole.

6.1 Component Testing

Component tests exist between unit tests and integration tests in terms of scope. While unit tests focus on individual units of code, component tests validate the interactions and integration between multiple units or components within a subsystem.

For the in-detail pass and fail results for each function and feature in our app, refer to appendix. Refer to appendix C for the

In summary, most features tested reached an expected outcome, with some superseding the original plan. Any changes to the personal or team data were reflected correctly in the UI and the Supabase database in real time. Only the

6.2 Integration Testing

6.3 Performance Testing

The initial testing

6.4 User Acceptance Testing

UAT or User Acceptance Testing ensures usability via real user interactions with the system. The application’s ease of use and overall functionality are scrutinised through human testers. Their feedback was collated through a general

Objective		

Chapter 7: Evaluation

This section outlines how aptly the requirements originally set out were adhered to, and includes any other analysis made on the final product.

7.1 Aims and Objectives

A quick rundown on the first chapter's broad goals

Objective	Achieved and to what degree?	Comments
Perform an extensive literature review	Yes,	Can be found in Chapter 2

7.1.1 Research Questions

7.2 Literature Review

The comprehensive literature review achieved its purpose of highlighting the areas of weight and pressure on amateur coaches and also showcasing the features that are already in use. Particularly, it emphasised the importance of a central system to manage and overview matches and games. It also highlighted the importance

Security and Usability

Stakeholder Feedback

After showing a working version of the app to the two first team captains, they stated that it was easy to use and they were happy with all the different features included.

For an intial prototype,

They included that

7.3 Requirement cases

Per the requirements table already added at appendix, the table below describes the efficacy when comparing our solution to the target. The ID prefix dictates whether it is a functional or non-functional requirement.

Req. ID	**Brief explanation**	**Outcome**
NF1		Executed
NF2		Partially -
NF3		Removed: No sensitive data is stored. All email and names are stored under a third party database which is GDPR compliant
NF4		

Chapter 8: Conclusion

8.1 Achievements

When I first started I had some understanding of React, however after completing this app, I feel very comfortable in explaining a lot of the topics. Furthermore, I have completed a wide arrange of new things from RLS to making sure

8.2 Challenges Faced and Personal Critiques

8.2.1 App Focus Pivot

From the original plan, there were two major deviations: The cross platform.

The original plan was to leverage an WhatsApp API for the communication aspect of the application. However, during the implementation I quickly realised the financial and technical limitations which were imposed by any API solution to bridge to WhatsApp.

8.2.2 Do Not Repeat Yourself

I know there were multiple points of improvement after a meticulous review of my code. I repeated myself numerous when creating buttons (Touchable Opacites). Now in the future, I will adhere more with the obvious DRY principle.

8.2.3 Pressable over Touchable Opacity

I had a lot of problems when dealing with navigation issues, and I tried multiple methods but if I had simplified my workflow to not include and followed more of the Expo traditional layouts with Pressable. It wasn't the point of error in most of the case it was memoisation, however Pressable in general allows a higher degree of control in terms of UI design.

8.2.4 Better file organisation

Provided that this was my first proper application using Expo Go, I used some outdate techniques and some random techniques to fix errors, leading to some modals being constructed inside the app/ directory and some other files being used in the components/ directory. This highlights a lack of organisational control over the tabs expo file based routing and I would not design the app like this again.

8.2.5 Code Modularity

I think for the purpose of this app, the manner in which I handled the different roles within the files was agreeable, however, if the project was to grow in nature, it would be much better to break down the different screens into their own separate views or at least split appropriately into different components.

8.3 App Future Improvements

For further app completion, it requires a couple features that have been pointed out and would significantly improve the experience. If multiple coaches could join the

Even though the

Potential Database Refactoring: Role-Specific Data Partitioning

Current State: A unified profiles table handles all user types (Coaches and Players).

Proposed Change: Split role-specific attributes into distinct tables to enforce stricter data integrity and reduce NULL values.

1. The Extension Pattern (Sidecar Tables)

Instead of moving everything, we would keep the core profiles table for authentication and basic info, then "extend" it with:

- player_details Table: Linked by profile_id. Contains position, shirt_number, injury_status.
- coach_details Table: Linked by profile_id. Contains license_type, experience_years, specialization.

2. Benefits of this Change

- Database Constraints: We can make the position field NOT NULL in the player table without affecting coaches.
- Cleanliness: Queries for team rosters won't return empty coach-specific columns.
- Scalability: If we add complex features (like detailed player scouting or coach certifications), the tables won't become unmanageable.

8.4 Final Words

This project was stressful but regardless I was proud of the toil. It made me deliberate purposefully on the app architecture and

References

Bokůvka, D. et al., 2025. Training load and fitness monitoring in Czech football. *Frontiers in Sports and Active Living,* Volume 7, p. 10.3389/fspor.2025.1513573.

Carrol, M. J., 2023. *A qualitative multiple case study exploration of the antecedents of the interpersonal behaviours of youth football coaches in Scotland based on a self-determination theory framework,* http://hdl.handle.net/1893/35263: University of Stirling.

Consultancy.uk, 2021. *English grassroots football adds billions in social and economic value.* [Online]
Available at: https://www.consultancy.uk/news/28926/english-grassroots-football-adds-billions-in-social-and-economic-value
[Accessed November 01 2026].

England[a], S., 2026. *Active Lives Adult Survey: Novemeber 2024-25,* London: Sport England Report.

Guardian, 2025. *This article is more than 10 months old Brentford FC received £3.23m in public money for research and development.* [Online]
Available at: https://www.theguardian.com/football/2025/feb/20/brentford-fc-public-money-for-research-and-development
[Accessed 28 October 2025].

Melnikovas, A., 2018. Towards an Explicit Research Methodology: Adapting Research Onion Model for Futures Studies. *Journal of Futures Studies,* 23(2)(10.6531/JFS.201812_23(2).0003), p. 29–44.

Ofcom, 2025. *Share of smartphone users in the United Kingdom (UK) from 2011 to 2024.* [Online]
Available at: https://www.statista.com/statistics/300398/smartphone-usage-in-the-united-kingdom
[Accessed 08 January 2026].

Product Plan, 2025. *MoSCoW Prioritization.* [Online]
Available at: https://www.productplan.com/glossary/moscow-prioritization/
[Accessed 02 January 2026].

Souaifi, M. et al., 2025. Artificial Intelligence in Sports Biomechanics: A Scoping Review on Wearable Technology, Motion Analysis, and Injury Prevention. *Bioengineering,* 12(8), p. 887.

Spond AS, 2026. *Google Play Spond Screenshots.* [Online]
Available at:
https://play.google.com/store/apps/details?id=com.spond.spond&hl=en_GB&pli=1
[Accessed 09 December 2025].

Spond, 2019. *The state of sports coaching in the UK 2019,*
https://static.spond.com/press/grassrootscoachingincrisis.pdf: Spond.

Sport England, 2025. *Number of people participating in football in England from 2015-16 to 2023-24.* [Online]

Available at: https://www.statista.com/statistics/934866/football-participation-uk/
[Accessed 07 January 2026].

Taylor, R., 2025. *Non-league and grassroots football: What is the state of play?.* [Online]
Available at: https://lordslibrary.parliament.uk/non-league-and-grassroots-football-what-is-the-state-of-play/
[Accessed 03 January 2026].

The Coaching Manual, 2023. *Who is it for - Coaches.* [Online]
Available at: https://www.thecoachingmanual.com/who-is-it-for/coaches
[Accessed 23 November 2025].

Zeylurt, M. a. A., 2016. Problems of amateur football: A qualitative methodological review. *International Journal of Social Sciences and Education Research,* 2(4), pp. 1148-1160.

Appendix A – Interview Questions

Below are the questions provided to target users, with the minimum number of responses need was selected to be 10. The reasoning behind the selection of questions and minimum threshold to use this primary research is explained in Ch2. Interview Questions.

Section 1: Background

1. How many players are typically in your team?
    - 11–15
    - 16–20
    - More than 20
2. How long have you been coaching or managing a sports team?
    - Less than 1 year
    - 1–3 years
    - 3–5 years
    - More than 5 years
3. How often does your team train or play matches?
    - Once a week
    - Twice a week
    - Three times or more
    - Seasonally / irregularly

Section 2: Current Team Management Practices

4. How do you currently schedule and communicate team activities (e.g. training, matches)?
    - WhatsApp group
    - Email
    - Dedicated app (e.g. Spond, Heja, TeamSnap)
    - Spreadsheet / manual methods
    - Other (please specify): _____
5. How do you track attendance or player availability?
    - Manually (e.g. spreadsheet, notes)
    - Messaging confirmations
    - Using an app

- I do not currently track this

6. How do you record or monitor player performance (e.g. goals, fitness, attendance consistency)?
    - I don't record performance
    - Basic manual records
    - Dedicated sports software
    - Custom solution (please describe): _____
7. How satisfied are you with your current process for managing your team?
    1. Very dissatisfied
    2. Dissatisfied
    3. Neutral
    4. Satisfied
    5. Very satisfied

Section 3: Challenges

9. What are the most challenging aspects of managing your team? (Select all that apply)
    - Player availability / attendance
    - Communication and coordination
    - Tracking performance / progress
    - Managing data (results, stats, etc.)
    - Motivation and engagement
    - Other (please specify): _____
10. How often do last-minute changes (e.g. cancellations, dropouts) affect your plans?

- Rarely
- Occasionally
- Often
- Very often

11. How much time per week do you spend on administrative or coordination tasks for your team?

- Less than 1 hour
- 1–3 hours
- 3–5 hours
- More than 5 hours

Section 4: Technology and AI

12. How comfortable are you using technology or apps to manage your team?
    1. Slightly comfortable
    2. Neutral
    3. Comfortable
    4. Very comfortable
13. Which of the following features would be most useful to you? (Select up to 3)

- Automated attendance tracking
- Performance insights (e.g. who is improving or inconsistent)
- Predictive availability (who is likely to attend)
- Fatigue detection/overtraining alerts
- Automated WhatsApp reminders
- AI-generated weekly summaries

14. How open would you be to using an AI assistant that analyses your team's data and provides coaching insights?
    1. Very closed
    2. Mildly closed
    3. Unsure
    4. Mildly open
    5. Very open
15. Would you trust AI-generated insights (e.g. "attendance predicted to drop next week due to low engagement") to inform your coaching decisions?
    1. Not at all
    2. Slightly
    3. Somewhat
    4. Mostly
    5. Completely
16. What concerns, if any, would you have about using an AI tool for managing your team?

- Privacy / data sharing
- Accuracy of predictions
- Complexity / learning curve
- Overreliance on technology

- None
- Other (please specify): _____

Section 5: Final Comments

Do you have any additional comments, frustrations, or ideas about how technology could make coaching easier?

Appendix B - Requirements

Non-functional Requirements

Pre-survey

ID	MoSCoW	Requirement	Measure	Notes
NF1	Must have	The application must have a graphic user interface.	Manual test	-
NF2	Must have	The app responds to user interaction within a reasonable amount of time	Latency test	-
NF4	Should have	All system data should be stored in a GDPR compliant manner	Audit	-
NF5	Should have	The complete application should be compatible with iOS and Android.	Build test	Necessary for accesibility
NF6	Should have	The backend should handle data processing even with inconsistent/small datasets	Manual test	-
NF7	Should have	Data Persistence: In the event of a crash, the system must ensure that no attendance or wellness data is lost.	Manual test	-

NF8	Should have	The system could integrate WhatsApp API to facilitate text messages.	Manual test	-
NF9	Could have	The mobile application must be lightweight to ensure it runs smoothly on older smartphone models	Build test	
NF10	Won't have	A web-based dashboard for coaches	-	Disregarded due to focus on mobile; time constraints

Post survey

ID	MoSCoW	Requirement	Measure	Notes
NF1	Must have	The application must have a graphic user interface.	Manual test	Retained: Necessary component
NF2	Must have	The app responds to user interaction within a reasonable amount of time	Latency test	Retained: Users agreed general usability of the application relies on decent response
NF4	Should have	All system data should be stored in a GDPR compliant manner	Audit	Retained.
NF5	**Should have**	The complete application should be compatible with iOS and Android.	Build test	**Upgraded:** More than 20% of users use a non iOS device. Base application should be modelled with accessibility as forefront.

NF6	Should have	The backend should handle data processing even with inconsistent/small datasets.	Manual test	Retained
NF7	**Must have**	Data Persistence: In the event of a crash, the system must ensure that no attendance or wellness data is lost.	Manual test	**Upgraded.**
NF8	Should have	The system could integrate WhatsApp API to facilitate text messages.	Manual test	-
NF9	**Should have**	The mobile application must be lightweight to ensure it runs smoothly on older smartphone models	Build test	**Upgraded:** Accessibility for the majority of phone
NF12	**Could have**	Application should render correctly across screens	Manual test	**Added:** A good UX experience is needed
NF10	Won't have	A web-based dashboard for coaches	-	Disregarded due to focus on mobile; time constraints
NF11	**Won't have**	The application will not support offline mode with local data sync	-	**Added:** Requires significant additional architecture; deferred.

Functional Requirements

Pre-survey

ID	MoSCoW	Requirement	Measure	Notes
F1	Must have	Coaches must be able to create a team profile and add/remove players.	Manual test	-
F2	Must have	A "Player Version" interface specifically for players to enter their data	Manual test	-
F3	Must have	Questionnaire data will be ‘sanitised’	Unit manual test	-
F4	Must have	Predictive model will use all given and pretrained data to produce team analytics on fatigue.	Manual	-
F5	Must have	The coach must be able to log and view attendance for every training session and match-day.	Manual	-
F6	Should have	A data entry interface for coaches to log injury/attendance and wellness metrics.	Manual Review	-
F7	Should have	Automated training reminders and match summaries sent via WhatsApp API	API Integration Test	Added feature for coach communcation

F8	Could have	Visual dashboard for coaches to display and analyse trends in player data.	User feedback	More coach support
F9	Could have	Create set messages to send to players for matches and events	User feedback	Is faster for when messaging players details
F10	Could have	A feature to generate a PDF summary of team reliability to be shared with committee	Manual	
F11	Won't have	Financial management or fee collection features.	-	-
F12	Won't have	The system will not generate training drills or session plans.	-	Out of scope;

Post survey

ID	MoSCoW	Requirement	Measure	Notes
F1	Must have	Coaches must be able to create a team profile and add/remove players.	Manual test	Retained: key part of application
F2	Must have	A "Player Version" interface specifically for players to enter their data	Manual test	Retained.
F3	Must have	Questionnaire data will be 'sanitised'	Unit manual test	Retained: high priority check for AI model

F4	Must have	Predictive model will use all given and pretrained data to produce team analytics on fatigue.	Manual	Retained.
F5	Must have	The coach must be able to log and view attendance for every training session and match-day.	Manual	Retained.
F6	**Could have**	A data entry interface for coaches to log injury/attendance and wellness metrics.	Manual Review	**Downgraded:** Would be good to implement, however not a necessary component.
F7	**Must have**	Automated training reminders and match summaries sent via WhatsApp API – FLAG change	API Integration Test	**Upgraded**. Increased urgency due to communication isolated as largest issue
F8	**Should have**	Visual dashboard for coaches to display and analyse trends in player data.	User feedback	**Upgraded** coach support
F9	Could have	Create set messages to send to players for matches and events	User feedback	Is faster for when messaging players details
F10	Could have	A feature to generate a PDF summary of team reliability to be shared with committee	Manual	Retained
F11	Won’t have	Financial management or fee collection features.	-	Retained: Out of scope.
F12	Won’t have	The system will not generate training drills or session plans.	-	Retained: Out of scope.

Appendix C – Component Testing

Feature	Followed procedure	Expected outcome	Passed
User authentication (for both players and coach unless specified)			
Create new account successfully; Registration	Enter valid email address and password, then click create account button	The app navigates to the main screen, and a new team is created correctly in our DB.	Yes
Registration with missing fields	Leave a field empty and click create account button	Alert message appears; input fields are reset.	Yes
Registration with weak password		Alert message highlighting the password minimum strength	No
Registration with existing email			
Registration without the team join code (Player)			
Registration with invalid email format			

Successful Login	Enter an existing user's credentials Click the sign in button		
Login with			
Forgot Password Functionality	Enter a valid email address	Successful reset ask message displayed; Email sent for password reset link.	Yes
Login with invalid email format		Appropriate error message	Yes
Main Screen (for both players and coach unless specified)			
Upcoming event details displayed	Navigate to home screen		
Upcoming event attendance in-detail breakdown (Coach)	Navigate to home screen Click on pressable		
Questionnaire Screen (for both players and coach unless specified)			

Be able to answer questionnaires (Player)			
Not be able to submit questionnaires of events that have not occurred yet (Player)		Should see a screen appropriately displaying a message explaining it must occur first.	
View questionnaire answers before enough questionnaires are answered (Coach)	Click on an event and click on the toggle button to the questionnaire screen	Should see a screen appropriately displaying a lack of user response	